

11.1 HD TASK (Machine Learning Research)

1. Research Problem and Background

Heart disease is a serious health problem, and finding possible signs early can help healthcare workers assess patients sooner. The research problem in this paper is whether patient health information can be used to predict the heart disease outcome recorded in a dataset. The information includes measurements such as age, blood pressure, cholesterol, and type of chest pain. The model's result is a prediction based on these records; it is not a medical diagnosis.

Researchers have used different machine learning models to study this problem. In the selected paper, the authors test six models: Logistic Regression, Naive Bayes, K-Nearest Neighbors (KNN), Decision Tree, Random Forest and XGBoost. Each model learns patterns from the patient records and makes its own prediction. The authors compare their results to see how well the different methods perform.

Earlier research has used individual machine learning models, feature selection, and ensemble methods to predict heart disease. The related-work section of the selected paper describes a hybrid method combining Random Forest with a linear model, which achieved 88.47% accuracy. It also describes a method combining chi-square feature selection with principal component analysis (PCA). That method reported accuracies of 98.7%, 99.0%, and 99.4% on three different datasets [1]. These results show that several approaches have been studied. However, differences in datasets and evaluation methods mean that these scores cannot be compared directly.

The research opportunity addressed by the selected paper is whether combining six classifiers can improve prediction performance compared with using each classifier separately. Different models may learn different patterns from the same patient information. The authors therefore use stacking to combine their predictions and evaluate the combined model against the individual models [1]. Earlier studies had already explored ensemble methods, so the contribution is the particular combination and evaluation investigated in this paper, rather than the first use of stacking for heart disease prediction.

The paper focuses on whether combining models can improve prediction results. To test this, the authors use a method called **stacking**. In stacking, several models make predictions, and another model learns how to combine those predictions into a final result. The authors compare this combined method with the six individual models. Their reported

stacking accuracy is **98.53%**. This gives us a published result to compare with our own reproduction in Part 1.

2. Dataset and Features

The dataset used in this project is the heart.csv file linked to the selected research paper. It contains **1,025 rows and 14 columns**. Thirteen columns contain patient information used to make predictions. The final column, called target, is the result the models try to predict. In this dataset, **526 rows have a target value of 1**, and **499 rows have a target value of 0**.

The input columns include age, sex, chest pain type (cp), resting blood pressure (trestbps), cholesterol (chol), fasting blood sugar (fbs), and maximum heart rate (thalach). The other inputs are restecg, exang, oldpeak, slope, ca, and thal. Some columns contain measurements, while others use numbers to represent categories. Before training the models, I checked the dataset and found **no missing values**.

I also found **723 duplicate rows** in the file. This matters because repeated records can affect how fairly a model is tested. I kept the dataset as used for my Part 1 reproduction and checked whether repeated records appeared in both the training and test sets. I discuss the effect on my results later in this report.

Dataset size, feature names, missing values, repeated rows, and target counts.

```
[2]
✓ 0s
import pandas as pd

data = pd.read_csv("heart.csv")

print("Rows and columns:", data.shape)
print("Columns:")
for column in data.columns:
    print("-", column)
print("Missing values:", data.isna().sum().sum())
print("Repeated rows:", data.duplicated().sum())
print("Target counts:")
print(data["target"].value_counts())
```

... Rows and columns: (1025, 14)
Columns:
- age
- sex
- cp
- trestbps
- chol
- fbs
- restecg
- thalach
- exang
- oldpeak
- slope
- ca
- thal
- target
Missing values: 0
Repeated rows: 723
Target counts:
target
1 526
0 499
Name: count, dtype: int64

3. Method and Implementation Choices

To reproduce the paper, I used the same dataset and tested the six individual models named by the authors: Logistic Regression, Naive Bayes, KNN, Decision Tree, Random Forest, and XGBoost. I also trained a stacking model that combines these models. This allowed me to compare the individual methods with the combined method.

I divided the 1,025 records into a **training set of 820 rows** and a **test set of 205 rows**. I used an 80/20 split, `random_state=42`, and stratification to keep the target classes in similar proportions in both sets. The models learned from the training set. I used the test set to measure their results.

Before training, I used `StandardScaler` to scale the numerical features: `age`, `trestbps`, `chol`, `thalach`, and `oldpeak`. I used `OneHotEncoder` to convert the categorical features—`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `ca`, and `thal`—into indicator columns. For the individual models, preprocessing was fitted on the training set and then applied to the test set. In the stacking model each base model had its own preprocessing pipeline. During five fold cross validation, each pipeline fitted its preprocessing only on that fold's training portion and applied it to the held out portion. This prevented the preprocessing from learning information from the held-out fold. The final stacking model was evaluated on the separate test set.

The paper does not give every setting needed to reproduce its experiment exactly. Therefore, the **80/20 split, random seed, preprocessing details and any model settings I selected** are my documented implementation choices. These choices may help explain differences between my results and the published results.

Component	Settings and search
Logistic Regression	C=0.3; max_iter=2000; search C: 0.03, 0.1, 0.3, 1, 3
SVM	C=1; probability=True; search C: 0.3, 1, 3; gamma: scale, 0.01
Random Forest	200 trees; min_samples_leaf=3; max_depth=5; search leaf: 2, 4, 8; depth: 3, 6
XGBoost	100 trees; max_depth=2; learning_rate=0.05; subsample=0.8; search trees: 50, 150; rate: 0.03, 0.1
Gaussian Naive Bayes	Default settings; no grid search
Final Logistic Regression	C=0.5; max_iter=1000; feature-only variant uses C=1.0
Parameter search	Three-fold stratified CV; selection metric AUC
Base-model predictions	Five-fold stratified CV with shuffle=True and seed 42
Threshold	Candidates 0.05 to 0.95 in steps of 0.01; highest candidate reaching training-validation recall of at least 0.90

Stratified 80/20 training and test split used in Part 1.

classes in similar proportions.

```
[4]
✓ Os from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training inputs:", X_train.shape)
print("Testing inputs:", X_test.shape)
print("Training target counts:")
print(y_train.value_counts())
print("Testing target counts:")
print(y_test.value_counts())
```

```
... Training inputs: (820, 13)
Testing inputs: (205, 13)
Training target counts:
target
1    421
0    399
Name: count, dtype: int64
Testing target counts:
target
1    105
0    100
Name: count, dtype: int64
```

4: Results and Comparison with the Paper

I tested the six individual models and the stacking model using the test set from Part 1. I measured each model using Accuracy, Precision, Recall, F1 Score and AUC. The following results show how the models performed in my notebook. I then compare them with the results reported in the paper.

Results from my Part 1 experiment.

Model	Accuracy	Precision	Recall	F1 Score	AUC
Logistic Regression	0.8732	0.8559	0.9048	0.8796	0.9445
Naive Bayes	0.8244	0.8108	0.8571	0.8333	0.8708
KNN	0.8634	0.8468	0.8952	0.8704	0.9698
Decision Tree	0.9854	1.0000	0.9714	0.9855	0.9857
Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000
XGBoost	1.0000	1.0000	1.0000	1.0000	1.0000
Stacking	1.0000	1.0000	1.0000	1.0000	1.0000

Results reported in the selected paper

Model	Accuracy	Precision	Recall	F1 Score	AUC
Logistic Regression	0.8439	0.8196	0.9090	0.8620	0.9204
Naive Bayes	0.8439	0.8250	0.9000	0.8608	0.9185
KNN	0.8585	0.8648	0.8727	0.8687	0.9304
Decision Tree	0.9268	0.9702	0.8909	0.9289	0.9702
Random Forest	0.9268	0.9130	0.9545	0.9333	0.9734
XGBoost	0.9073	0.9174	0.9090	0.9132	0.9830
Stacking	0.9853	1.0000	0.9727	0.9861	0.9880

My stacking model scored 1.0000 on all five measures, while the paper reported an accuracy of 0.9853 and an AUC of 0.9880 for stacking. My Random Forest and XGBoost models also scored 1.0000 on all five measures, which is higher than the paper's results for these models. The other models showed mixed differences. For example, my Logistic Regression accuracy was 0.8732, compared with 0.8439 in the paper, but my Naive Bayes accuracy was lower. These differences may be related to the data split and the model settings used in each experiment. I explain one important issue with my test split in the next paragraph.

```
[5]
✓ 0s train_rows = X_train.copy()
      train_rows["target"] = y_train

      test_rows = X_test.copy()
      test_rows["target"] = y_test

      shared_rows = test_rows.merge(train_rows.drop_duplicates(), how="inner")

      print("Test rows also found in training:", len(shared_rows))
      print("Total test rows:", len(test_rows))

... Test rows also found in training: 202
     Total test rows: 205
```

This overlap helps explain why some models received perfect scores. Because copies of many test rows were already in the training data, these results may be too high for new, unseen records. The overlap is a limitation of my Part 1 test. I will use a split that keeps identical records together in Part 2.

Overall, I recreated the paper's six individual models and its stacking approach using the same heart disease dataset. My stacking model achieved perfect scores on this test split, while the paper reported lower scores. However, many identical records appeared in both my training and test sets. Therefore, the perfect scores do not show how well the model would work on new patients. I will address this limitation in Part 2.

Part 2 Stacking with Duplicate Safe Evaluation

1 Limitation and Research Motivation

My Part 1 reproduction contained 723 duplicate rows among 1,025 records. After splitting 202 of the 205 test rows also appeared in training. This overlap may make the results too optimistic. This finding concerns my reproduction and does not establish overlap in the published experiment [1]. The dataset contains 302 unique feature records, without patient identifiers these cannot be confirmed as 302 different patients.

My solution prevents exact duplicate overlap and investigates whether derived features, regularisation, parameter tuning and a training selected threshold can improve prediction under a cleaner evaluation. I focus on recall while also measuring the cost in precision and specificity. This is a dataset classification study not a validated clinical screening tool.

2 Proposed Method

I removed exact duplicate rows and checked whether identical feature records had conflicting labels. The check found zero conflicts. The remaining data contained 164 positive and 138 negative records. All compared methods used the same outer evaluation folds.

I added three exploratory features: $hr_ratio = thalach / (220 - age)$, $ca_any = 1$ when ca is greater than zero and $oldpeak_slope = oldpeak$ when $slope$ equals 2 and zero otherwise. These features represent derived relationships in the available inputs. Their clinical validity is not established by this experiment.

The modified stack combines Logistic Regression a support vector machine, Random Forest, XGBoost and Gaussian Naive Bayes. A regularised Logistic Regression final model combines their out of fold probabilities. I compare variants with derived features a modified base model set, tuning and a sensitivity focused threshold. Some variants change multiple settings so the comparisons do not isolate the effect of every individual component.

3 Preprocessing Settings and Validation

StandardScaler prepares age, trestbps, chol, thalach and oldpeak. OneHotEncoder prepares sex, cp, fbs, restecg, exang, slope, ca and thal. The derived ratio and interaction enter the numerical pipeline while ca_any enters the categorical pipeline. Each base model pipeline fits its preprocessing within its training portion.

I used repeated stratified five fold cross validation with five repeats and random seed 42 giving 25 outer test folds. Each record is evaluated once per repeat. Outer test folds are excluded from fitting, parameter search and threshold selection. Fold standard deviations describe variability, they are not confidence intervals.

Model	Settings used
Logistic Regression	max_iter=1000, random_state=42
Naive Bayes	GaussianNB with default settings
KNN	5 neighbours
Decision Tree	Default settings, random_state=42
Random Forest	100 trees, random_state=42
XGBoost	100 trees, random_state=42, eval_metric="logloss"
Stacking	Six base models, Logistic Regression as the final model, five-fold cross-validation, prediction probabilities used to combine models

The current implementation selects base-model parameters on the entire outer training fold before generating base model out of fold predictions. Threshold selection cross validates the final model on an existing prediction matrix rather than refitting the complete stack. Therefore, the internal procedure is not fully nested and its training validation estimates may be optimistic. The outer test evaluation still excludes those test records from these fitting steps. A target sensitivity of 90% during threshold selection does not guarantee 90% on outer test data.

4 Results and Comparison

The following values are saved mean results across the 25 outer test folds in my current notebook. They are reported to four decimal places. The baseline stack retains the Part 1 architecture but is retrained under the clean protocol.

Method	Accuracy	Precision	Recall	F1	AUC
Logistic Regression	0.8450	0.8446	0.8828	0.8607	0.9135
Random Forest	0.8213	0.8317	0.8488	0.8370	0.9085
Part 1 style stack on clean data	0.8324	0.8392	0.8633	0.8482	0.9079
A1 derived features	0.8364	0.8394	0.8694	0.8514	0.9091
A2 modified regularised base set	0.8424	0.8384	0.8852	0.8588	0.9135
A3 parameter tuning	0.8390	0.8355	0.8828	0.8560	0.9142
Full proposed method	0.8371	0.8233	0.8975	0.8565	0.9142

Compared with the clean baseline stack the full method increased mean accuracy by 0.47 percentage points recall by 3.42 points F1 by 0.83 points and AUC by 0.0063 based on rounded values. Precision fell by 1.59 points. Mean specificity fell from about 0.796 to 0.765. The method therefore found a larger proportion of positive records while also producing more false positives

The full method was not best on every measure. Logistic Regression achieved higher accuracy, precision and F1. The A2 variant also had higher accuracy than the full method. Adding tuning and threshold selection did not consistently improve every metric. The full method reached mean recall of about 89.75%, below the training-selection target of 90%.

Experiment	Accuracy	Precision	Recall	F1	AUC
Paper published results [1]	0.9853	1.0000	0.9727	0.9861	0.9880
Part 1 reproduction	1.0000	1.0000	1.0000	1.0000	1.0000
Clean baseline stack	0.8324	0.8392	0.8633	0.8482	0.9079
Full proposed method	0.8371	0.8233	0.8975	0.8565	0.9142

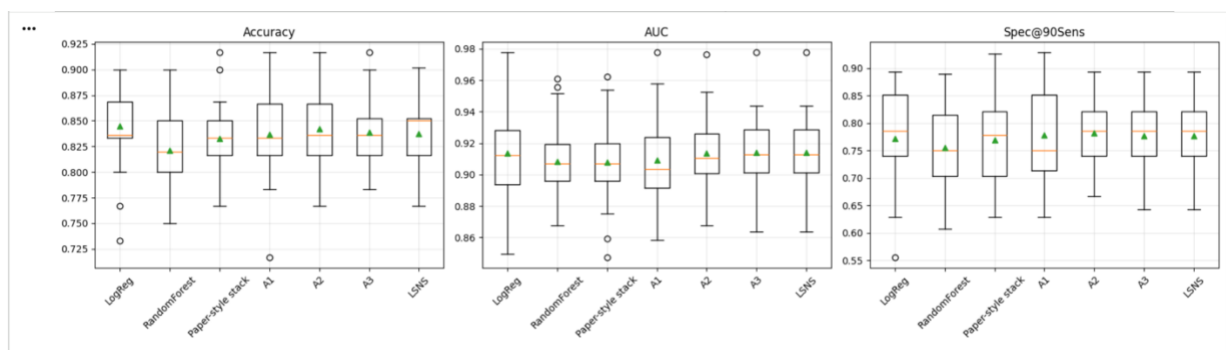
Published results and the Part 1 split are not directly comparable with repeated validation on unique records. Differences include evaluation records training size and protocol. The lower clean scores do not establish that the authors used leakage or that duplicate overlap alone explains the full difference. The clean baseline and proposed variants can be compared more directly because they use identical outer folds.

5 Statistical Evidence and Figures

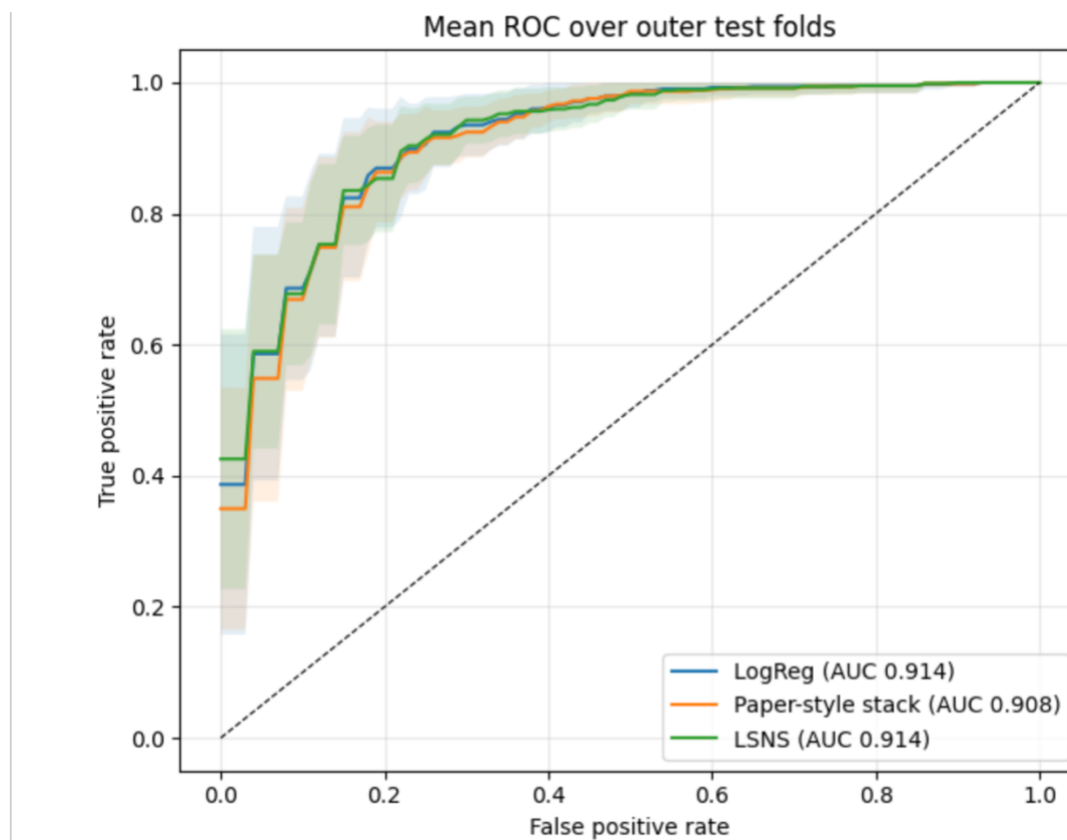
I report a Wilcoxon signed rank test as an exploratory comparison and a corrected resampled t test to account approximately for dependence between repeated folds. Holm correction is applied across model comparisons separately for each tested metric. Wilcoxon alone does not resolve fold dependence. For the full method against the clean baseline stack, corrected adjusted p values were 1.0000 for accuracy, F1 and AUC and 0.9609 for recall. These results do not establish statistically significant improvement at the 0.05 level. They also do not prove that the methods are equivalent.

The notebook includes fold distribution plots mean ROC curves, calibration plots, variant comparisons, confusion matrices and held out permutation importance. Specificity at 90% sensitivity calculated from test ROC curves is a descriptive operating point measure, not an independently validated deployment threshold. Pooled repeated validation predictions include each record multiple times. Permutation importance describes model behavior and does not establish medical causation.

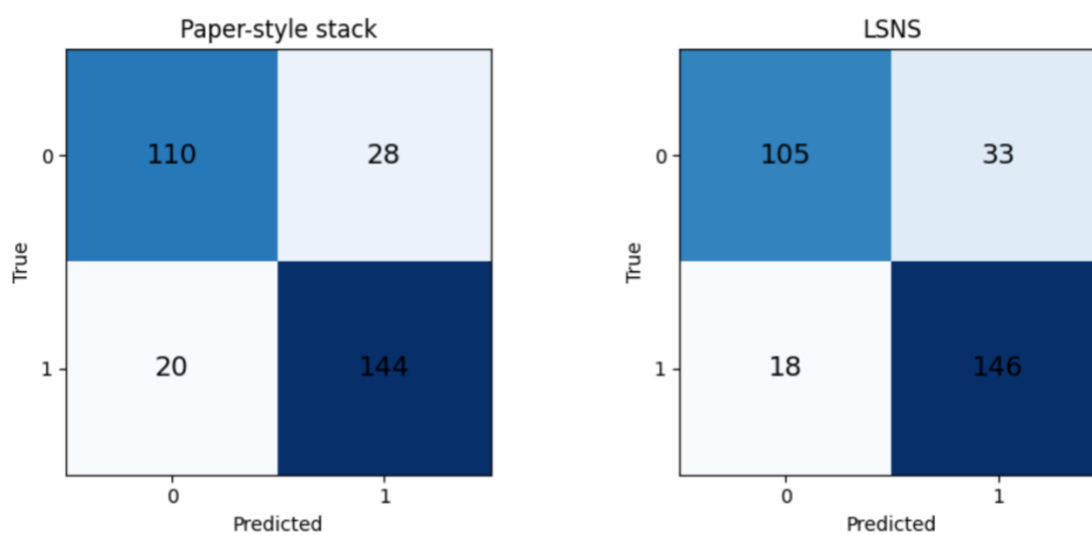
Fold variability for accuracy AUC and specificity



Fold variability for accuracy AUC and specificity



Mean ROC curves across outer test folds



6 Limitations and Conclusion

This study has only 302 unique records no verified patient identifiers and no external validation dataset. Internal parameter and threshold validation have the limitations described above. The exploratory features need stronger domain justification and the variant ladder changes more than one setting in some steps. Repeated outer evaluation improves the evidence compared with one split but repeated folds are dependent and cannot substitute for external validation.

The main contribution is prevention of exact duplicate overlap and a broader, reproducible comparison of stacking variants. The full method shows a small average gain over the clean baseline stack and higher recall at the cost of precision and specificity. Corrected statistical tests do not establish superiority. These findings support a cautious conclusion rather than a claim that the method is clinically effective or better than the published paper.

Generative AI Acknowledgement

I used ChatGPT to understand the requirements of the task to develop and revise the Python code to interpret the results of the experiments and write the explanations for the report and the video presentation. AI assistance also helped to organize the notebook and to improve the clarity of my writing. I accept responsibility for the submitted work (checked code, results, references and explanations).

References

[1] M. Bhagat, A. Sharma, and P. Agarwal, “An efficient stacking-based ensemble technique for early heart attack prediction,” *Multimedia Tools and Applications*, vol. 84, pp. 36351–36375, 2025, doi: 10.1007/s11042-024-19293-7.

Video demonstration

<https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=8a0f373d-d08c-4aaa-80cd-b4d7006a0588>

GitHub link

[https://github.com/ravishanwedisinghe/11.1-HD-TASK-Machine-Learning-Research-
/tree/main](https://github.com/ravishanwedisinghe/11.1-HD-TASK-Machine-Learning-Research-/tree/main)